

Project 2 — One-Time Pad: Bob Decryption

This notebook implements Bob's portion of the one-time-pad exercise.

Bob:

1. Reads the ciphertext from a hexadecimal text file.
2. Reads the key from a hexadecimal text file.
3. Converts both values to bytes.
4. XORs the ciphertext with the key.
5. Decodes and displays the recovered plaintext.

```
In [1]: from pathlib import Path

def xor_bytes(message: bytes, key: bytes) -> bytes:
    if len(message) != len(key):
        raise ValueError("Ciphertext and key must be the same length.")

    return bytes(m ^ k for m, k in zip(message, key))

def read_hex_file(filename: str) -> bytes:
    hex_text = Path(filename).read_text(encoding="utf-8").strip()
    return bytes.fromhex(hex_text)

def decrypt(ciphertext: bytes, key: bytes) -> str:
    plaintext_bytes = xor_bytes(ciphertext, key)
    return plaintext_bytes.decode("utf-8")
```

Bob-Side Test

Until Alice's implementation is integrated, the following cell creates a known ciphertext/key pair so that Bob's decryption logic can be tested independently.

```
In [2]: test_plaintext = "Hello from Alice!".encode("utf-8")

test_key = bytes([
    0x37, 0x91, 0x42, 0xAC,
    0x18, 0xDD, 0x73, 0x05,
    0xE1, 0x66, 0x29, 0xB4,
    0x53, 0x80, 0x19, 0xCE,
    0x44
])

test_ciphertext = xor_bytes(test_plaintext, test_key)

Path("ciphertext.txt").write_text(test_ciphertext.hex())
Path("key.txt").write_text(test_key.hex())
```

Out[2]: 34

```
In [3]: ciphertext = read_hex_file("ciphertext.txt")
key = read_hex_file("key.txt")

plaintext = decrypt(ciphertext, key)

print("Recovered plaintext:")
print(plaintext)
```

Recovered plaintext:
Hello from Alice!

In []: